

T.N.S.V.T

Wallet & Torneos con dinero real

Arquitectura completa para monetizar la comunidad

12 amigos ya estan listos para poner plata. Arrancamos.

Fase	Stack	Tiempo	Status
1. MVP manual	Admin panel + DB	1-2 días	■ ARRANCA HOY
2. MercadoPago AR	API MP + webhooks	2-3 días	Semana 2
3. Binance Pay	binance-connector-php	2-3 días	Semana 3
4. Crypto directo	Self-custody multi-chain	3-5 días	Mes 2
5. Stripe (cards)	stripe/stripe-php	1 dia	Cuando haga falta
6. Wise (bancos)	transferwise/api	1 dia	Cuando haga falta

Documento generado: 17/06/2026 16:24
Version: 1.0 - Plan inicial
Autor: TNSVT Dev
Stack: Symfony 7 + PHP 8.4 + SQLite + Capacitor 6 + Vanilla JS

Indice

1. Vision general y fases
2. Fase 1 - MVP manual (lo que arrancamos)
3. Schema de base de datos
4. Endpoints de la API
5. Frontend TNSVT (sidebar + tabs)
6. Frontend Game (s-torneos refactor)
7. Cron y auto-close
8. Admin panel completo
9. Conversion ARS-USD (live rate)
10. Fase 2 - MercadoPago Argentina
11. Fase 3 - Binance Pay
12. Fase 4 - Otros paises
13. Compliance & legal en Argentina
14. Plan por etapas (4-6 stages)
15. Costos, ROI y proyeccion

1. Vision general y fases

El objetivo

Convertir T.N.S.V.T de un juego didactico gratis a una plataforma con economia interna donde los traders pueden competir entre si por dinero real. La mecanica es simple: el admin crea un torneo con un entry fee (ej. \$5 USD), los users se suman, operan una semana en el Portfolio virtual, y el que mas PnL genera se lleva el pozo.

El truco: Portfolio mode ya esta listo

TNSVT ya tiene un Portfolio demo con \$50k virtuales, posiciones LONG/SHORT, apalancamiento, stop loss, take profit, y ahora precios reales de CoinGecko. Los torneos reutilizan esta infraestructura - no hay que inventar nada nuevo, solo agregar:

- 1. Un wallet virtual por user (tabla users.wallet_balance)
- 2. Un sistema de torneos con entry fee + prize pool
- 3. Una UI para depositar plata real (MP/Binance/Crypto)
- 4. Una UI para ver leaderboard en vivo durante la semana

Fases de implementacion

Fase	Que resuelve	Stack	Tiempo	Costo	Complejidad
1. MVP	Lanzar YA con 12 amigos	Admin panel + DB	1-2 dias	\$0	■ Baja
2. MP AR	Auto-deposito en ARS	mercadopago/sdk-php	2-3 dias	\$0 + fees MP	■ Media
3. Binance	USDT/USDC global	binance-connector-php	2-3 dias	\$0 (0% fee)	■ Media
4. Crypto	USDT ERC20/TRC20/BEP20	web3.php + alchemy	3-5 dias	\$0-50/mes RPCs	■ Alta
5. Stripe	Cards internacionales	stripe/stripe-php	1 dia	0 + 2.9% fee	■ Media
6. Wise	Bancos EU/UK	transferwise/api	1 dia	0 + 0.5% fee	■ Media

Por que Fase 1 manual primero

Con 12 amigos, vos podes recibir un MP de \$5.000 ARS, mirar tu cuenta, y acreditarles el balance a mano en 30 segundos. Es mas seguro, mas rapido de implementar, y no requiere ninguna API externa. Cuando tengas 50+ users o quieras escalar, swap a Fase 2 con MP API. El codigo de Fase 1 se mantiene - solo cambia el "como se acredita" (manual vs auto).

2. Fase 1 - MVP manual (la que arrancamos)

Flujo end-to-end de un user

Dia 0 (deposito):

1. Juan te manda \$5.000 ARS por MP
2. Vos vas a /admin/wallet en tu panel privado
3. Pones code "juanp01", monto \$5 USD (equiv a \$5.000 ARS hoy), boton "Acreditar"
4. Se crea wallet_transaction type=deposit, status=confirmed
5. Su balance sube a \$5 USD

Dia 0 (entrar a torneo):

1. Juan ve el torneo activo "Semanal \$5" en s-torneos
2. Click "Unirme \$5" - el sistema le descuenta \$5 de su wallet
3. Se crea tournament_entry, captura starting_equity = balance + open_pnl actual
4. Aparece en el leaderboard del torneo

Dia 1-7 (compete):

1. Juan opera en Portfolio, gana/perdin PnL
2. Cada tick actualiza su pnl_pct = (current_equity - starting) / starting * 100
3. Leaderboard muestra rank en vivo

Dia 7 (cierre):

1. Vos (o cron automatico) cierra el torneo
2. Top 1 recibe 60% del prize pool, top 2 recibe 30%, top 3 recibe 10%
3. Se acredita a sus wallets automaticamente
4. Vos les mandas el MP/transfer a mano
5. Torneo queda en historial con todos los ranks y payouts

Como se calcula el rank

Cada participante tiene un **starting_equity** capturado al momento del join. Mientras el torneo esta activo, el sistema mira en vivo el **current_equity** = balance + open_pnl de su Portfolio. El **pnl_pct** = (current - starting) / starting * 100. El ranking se ordena por pnl_pct descendente. Si un user no opera durante la semana, su pnl es 0 y queda ultimo. Si todos pierden plata, el que menos perdio gana.

Que pasa si el torneo no llega a 3 participantes

Si hay 1 participante: se cancela y se devuelve el entry fee. Si hay 2: se redistribuye 60/40 al top 1/2. Si hay 3+: 60/30/10 como dice prize_distribution.

Que pasa si un participante hace fraude

Como todo es virtual y el admin controla los credits, no hay vector de fraude del lado del user. Lo unico que podria pasar es que alguien abuse de la confianza (jugar y no pagar, o pagar y no recibir payout). Por eso 12 amigos es manejable - si uno se manda una cagada, lo bloqueas. Si creces a 100+ users desconocidos, ahi si necesitas KYC + reputacion.

3. Schema de base de datos

Migracion SQL completa (SQLite)

Toda la migracion se hace via Doctrine Migrations. Aca esta el SQL equivalente para referencia:

```
-- 1. Agregar wallet_balance a users
ALTER TABLE users ADD COLUMN wallet_balance DECIMAL(12,2) DEFAULT 0.00 NOT NULL;

-- 2. Wallet transactions
CREATE TABLE wallet_transactions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type VARCHAR(20) NOT NULL, -- 'deposit'|'entry_fee'|'payout'|'refund'|'withdraw'
  amount DECIMAL(12,2) NOT NULL, -- +credita / -debit
  currency VARCHAR(8) DEFAULT 'USD' NOT NULL,
  ref_tournament_id INT NULL,
  ref_payment_id VARCHAR(100) NULL, -- MP payment_id o Binance prepayId
  ref_payment_method VARCHAR(20) NULL, -- 'mp'|'binance'|'crypto'|'manual'
  status VARCHAR(20) DEFAULT 'confirmed' NOT NULL, --
  'pending'|'confirmed'|'rejected'|'refunded'
  notes TEXT,
  created_at DATETIME NOT NULL,
  confirmed_at DATETIME NULL,
  confirmed_by INT NULL REFERENCES users(id) -- admin que acredito (si es manual)
);
CREATE INDEX idx_wtx_user ON wallet_transactions(user_id);
CREATE INDEX idx_wtx_tournament ON wallet_transactions(ref_tournament_id);
CREATE INDEX idx_wtx_status ON wallet_transactions(status);
CREATE INDEX idx_wtx_created ON wallet_transactions(created_at);

-- 3. Tournaments
CREATE TABLE tournaments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  entry_fee DECIMAL(10,2) NOT NULL,
  prize_pool DECIMAL(12,2) DEFAULT 0.00 NOT NULL,
  prize_distribution VARCHAR(20) DEFAULT '60,30,10' NOT NULL, -- pct para 1/2/3
  start_date DATETIME NOT NULL,
  end_date DATETIME NOT NULL,
  status VARCHAR(20) DEFAULT 'pending' NOT NULL, --
  'pending'|'active'|'closed'|'finished'|'cancelled'
  max_players INT DEFAULT 100 NOT NULL,
  min_players INT DEFAULT 2 NOT NULL,
  created_by INT NOT NULL REFERENCES users(id),
  created_at DATETIME NOT NULL,
  finished_at DATETIME NULL
);
CREATE INDEX idx_trn_status ON tournaments(status);
CREATE INDEX idx_trn_end_date ON tournaments(end_date);
CREATE INDEX idx_trn_active ON tournaments(status, end_date);

-- 4. Tournament entries
CREATE TABLE tournament_entries (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  tournament_id INT NOT NULL REFERENCES tournaments(id) ON DELETE CASCADE,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  starting_equity DECIMAL(14,4) NOT NULL, -- PORT.balance + open_pnl al join
  final_equity DECIMAL(14,4) NULL,
  pnl_usd DECIMAL(14,4) NULL, -- final - starting
```

```
    pnl_pct DECIMAL(10,6) NULL, -- pnl_usd / starting * 100
    final_rank INT NULL,
    payout_amount DECIMAL(12,2) NULL,
    status VARCHAR(20) DEFAULT 'active' NOT NULL, -- 'active'|'finished'|'disqualified'
    joined_at DATETIME NOT NULL,
    finalized_at DATETIME NULL,
    UNIQUE(tournament_id, user_id)
);
CREATE INDEX idx_te_tournament ON tournament_entries(tournament_id);
CREATE INDEX idx_te_user ON tournament_entries(user_id);
CREATE INDEX idx_te_status ON tournament_entries(status);
CREATE INDEX idx_te_pnl ON tournament_entries(tournament_id, pnl_pct DESC);
```

Entidades Doctrine

Cada tabla tiene su entity PHP correspondiente (src/Entity/) con relaciones, getters/setters, y metodos de negocio:

User (+ \$walletBalance, getWalletBalance(), addToWallet(\$amount), hasBalance(\$min))

WalletTransaction (const TYPE_DEPOSIT, TYPE_ENTRY_FEE, TYPE_PAYOUT, etc, getFormattedAmount())

Tournament (const STATUS_*, isActive(), getDaysRemaining(), getCurrentPrizePool(), getParticipants())

TournamentEntry (computeCurrentPnl(\$currentEquity), isActive())

4. Endpoints de la API

Wallet

Metodo	Path	Auth	Descripcion
GET	/api/wallet/balance	sesion o X-Game-Code	Devuelve el balance USD del user actual
GET	/api/wallet/transactions	sesion o X-Game-Code	Lista de transacciones del user
GET	/api/wallet/rates	publico	Cotizacion ARS-USD blue/oficial/mep/tarjeta
POST	/api/wallet/withdraw	sesion o X-Game-Code	Solicita retiro (queda pending hasta que admin confirme)

Admin wallet

Metodo	Path	Auth	Descripcion
POST	/api/admin/wallet/credit	admin password	Acredita USD a un user (uso: confirmaste MP/Binance)
POST	/api/admin/wallet/debit	admin password	Debita USD de un user (uso: pagaste un payout)
GET	/api/admin/wallet/pending	admin password	Lista de withdraws pendientes
POST	/api/admin/wallet/withdraw/{id}/approve	admin password	Aprueba un withdraw
POST	/api/admin/wallet/withdraw/{id}/reject	admin password	Rechaza un withdraw (devuelve el monto)

Torneos

Metodo	Path	Auth	Descripcion
GET	/api/tournaments/active	publico	Lista de torneos activos (status=active y end_date > now)
GET	/api/tournaments/{id}	publico	Info de un torneo + lista de participantes
POST	/api/tournaments/{id}/join	user	Une al user al torneo, descuenta entry_fee del wallet
GET	/api/tournaments/{id}/leaderboard	publico	Leaderboard en vivo ordenado por pnl_pct
GET	/api/tournaments/my	user	Historial de torneos del user (activos, finalizados)
POST	/api/admin/tournaments/create	admin	Crea un nuevo torneo
POST	/api/admin/tournaments/{id}/close	admin	Cierra torneo, calcula ganadores, distribuye prize pool
POST	/api/admin/tournaments/{id}/cancel	admin	Cancela torneo (ej: si no llega al minimo)

Detalles de /api/tournaments/{id}/join

Request body (vacío, usa sesión o X-Game-Code):

Response 200:

```
{
  "tournament_entry_id": 42,
  "tournament_id": 7,
  "starting_equity": 50000.0000,
  "wallet_balance_after": 45.00,
  "current_rank": 4
}
```

Errores posibles:

400: tournament_not_active, wallet_insufficient (mostrar cuanto falta), already_joined, tournament_full

404: tournament_not_found

401: no autenticado

Detalles de /api/admin/tournaments/create

Request body:

```
{
  "name": "Semanal Express",
  "description": "1 semana - BTC/ETH/SOL - el mejor trader se lleva el pozo",
  "entry_fee": 5.00,
  "duration_days": 7,
  "max_players": 50,
  "min_players": 3,
  "prize_distribution": "60,30,10",
  "start_now": true // si false, tomar start_date del body
}
```

Response 200:

```
{
  "tournament_id": 7,
  "name": "Semanal Express",
  "status": "active",
  "start_date": "2026-06-17T18:00:00Z",
  "end_date": "2026-06-24T18:00:00Z",
  "entry_fee": 5.00,
  "prize_pool": 0.00,
  "participants": 0
}
```


5. Frontend TNSVT (sidebar + tabs)

Sidebar nuevo

Se agregan 2 botones al sidebar de TNSVT (debajo de Chat de Ejecutores):

- Mi Wallet
- Torneos

El admin tiene ademas:

- Admin (existente - le agregamos sub-tabs de Wallet + Torneos)

Tab Wallet

Layout mobile-first con 4 bloques:

1. **Balance card** (header): \$XX.XX USD, equivalente en ARS (live rate), boton "Depositar"
2. **Acciones rapidas**: Depositar / Retirar / Historial
3. **Metodos de deposito** (segun fase):
 - Fase 1: Solo "Pedile al admin que acredite" + tu code visible para copiar
 - Fase 2: + MercadoPago (boton amarillo, abre checkout)
 - Fase 3: + Binance Pay (boton amarillo, abre QR)
 - Fase 4: + Crypto directo (boton, muestra address + QR)
4. **Transacciones recientes**: lista de los ultimos 20 movimientos con tipo, monto, fecha

Tab Torneos (user)

Tabs internos: **Activos** | **Mis torneos** | **Historial**

Activos: grid de cards con torneos disponibles. Cada card muestra: nombre, entry fee, prize pool actual, participantes, tiempo restante, boton "Unirme \$X"

Mis torneos: cards con tus entries. Cada card muestra: nombre, mi rank, mi pnl % actual, tiempo restante, link "Ir al Portfolio"

Historial: tabla con torneos pasados: nombre, mi rank final, mi pnl, payout recibido, fecha

Admin panel - sub-tabs nuevos

Dentro del tab Admin existente, se agregan:

Sub-tab Wallet:

- Form: code de user + monto USD + notas -> Acreditar
- Form: code de user + monto USD + notas -> Debitar (payout manual)
- Lista de transacciones pendientes de aprobar (withdraws)
- Auditoria: ultimas 100 transacciones

Sub-tab Torneos:

- Form: crear torneo (nombre, fee, duracion, distribucion)
- Lista de torneos: filtrable por status, con acciones (cerrar/cancelar)
- Auditoria: todos los payouts realizados

6. Frontend Game (s-torneos refactor)

La pantalla s-torneos del Game ya existe pero es hardcodeada. La refactorizamos para que traiga data real del backend:

Estructura nueva del s-torneos

Header: titulo grande "■ Torneos" + subtítulo "Compíte por dinero real"

Mi estado: card con mi rank actual, mi pnl % en mi torneo activo, tiempo restante

Sub-tabs: Activos | Mis torneos | Historial

Grid de cards: 2 columnas, cada card con info del torneo

Acción al clickear card: modal con detalles + leaderboard inline + botón Unirme/Ver mi posición

Sincronización con backend

El Game ya tiene TNSVT_CONFIG con serverUrl y code. Solo hay que agregar:

tournamentsInit(): al abrir s-torneos, hace GET /api/tournaments/active con X-Game-Code

tournamentJoin(id): POST /api/tournaments/{id}/join, actualiza balance y lista

tournamentLeaderboard(id): GET /api/tournaments/{id}/leaderboard cada 30s mientras esta abierto

Realtime: el leaderboard se actualiza cada 30s con polling (suficiente para 12 users)

Edge case: user sin code TNSVT

El Game es standalone - si el user no configuro su code TNSVT en Sync TNSVT, los botones de torneo muestran un mensaje "Primero configura tu código TNSVT en Perfil -> Sync TNSVT" y link al tab correspondiente. No bloqueamos el juego, solo el feature de torneos.

7. Cron y auto-close

Symfony Scheduler

Usamos symfony/scheduler (incluido en symfony/scheduler-bundle) para correr tareas automaticas:

- **Cada 1 minuto:** actualizar status de torneos (pending -> active cuando llega start_date, active -> closed cuando llega end_date)
- **Cada 5 minutos:** refrescar leaderboard de torneos activos en cache (opcional)
- **Cada 1 hora:** refrescar rate ARS-USD de dolarapi.com (cache 1h)
- **Cuando un torneo se cierra:** calcular ganadores, distribuir prize pool, marcar como finished

Comando artisan

Se crea un comando "tournaments:process" que:

1. Busca torneos con status=active y end_date <= now
2. Para cada uno, cambia status a closed
3. Calcula final_equity, pnl_pct, final_rank para cada entry
4. Ordena por pnl_pct DESC, asigna rank
5. Distribuye prize pool segun prize_distribution
6. Crea wallet_transactions type=payout para los winners
7. Suma payout_amount al wallet_balance de cada winner
8. Cambia status a finished + set finished_at

Comando "rates:refresh"

Hace GET a <https://dolarapi.com/v1/dolares/{blue,oficial,mep,tarjeta}> y guarda en cache (tabla rates_cache o APCu). Si falla, usa el ultimo valor conocido y loggea warning.

8. Admin panel completo

Autenticacion de admin

Reusamos el patron existente en TNSVT: el admin tiene un header "X-Admin-Password" con el valor "TNSVT-2026-CristoRey!" (mismo que ya usas para login admin). El backend valida en cada endpoint /api/admin/*.

Alternativa futura: tabla admins con sus propios users. Para 1 admin (vos), el header es suficiente.

Pantalla admin - Wallet

Form de acreditar:

- Input: user code (autocomplete desde lista de users activos)
- Input: monto USD (default 0, validacion positivo)
- Select: payment method (manual_mp, manual_binance, manual_crypto, gift, other)
- Textarea: notas (ej: "Confirmado por MP operacion 12345")
- Boton: "Acreditar \$X.XX USD a {code}"

Tabla de transacciones recientes con filtros: por user, por tipo, por fecha.
Paginada (50 por pagina).

Pantalla admin - Torneos

Form de crear:

- Input: nombre
- Textarea: descripcion
- Input: entry fee USD (default 5)
- Select: duracion (1d, 3d, 7d, 14d, 30d)
- Input: max players (default 100)
- Input: min players (default 2)
- Select: prize distribution (50/30/20, 60/30/10, 100)
- Checkbox: start now vs fecha custom
- Boton: "Crear torneo"

Lista de torneos con acciones: ver leaderboard, cerrar, cancelar, ver detalle.

Auditoria

Toda accion admin se loggea en wallet_transactions y tournament_entries con confirmed_by y created_at. Esto te da un trail completo de quien acredita que, cuando, y por que. Para 12 amigos es overkill, pero cuando crezcas te salva de disputas.

9. Conversion ARS-USD (live rate)

Por que importa

MercadoPago Argentina cobra en pesos. Vos necesitas saber cuanto USD equivale para acreditar el wallet del user. Hay 3 cotizaciones relevantes:

- **Oficial:** el del BCRA, siempre mas bajo
- **Blue:** el del mercado paralelo, siempre mas alto (el que usa la gente)
- **Tarjeta:** el que cobra MP cuando pagás con tarjeta, incluye impuestos

Recomendacion: **usar Blue** como default (es lo que la gente usa en la calle). El user ve "USD 10 = ARS 12.000 (blue)" antes de pagar, y vos le acreditas los 10 USD exactos.

API publica: dolarapi.com

Endpoint: `https://dolarapi.com/v1/dolares/blue`

Response:

```
{
  "moneda": "USD",
  "casa": "blue",
  "nombre": "Blue",
  "compra": 1180,
  "venta": 1200,
  "fecha": "2026-06-17T18:00:00.000Z"
}
```

Otros endpoints: `/oficial`, `/mep`, `/ccl`, `/tarjeta`, `/cripto`. Sin autenticacion, sin rate limit agresivo, gratis, ~99.9% uptime. Refresh: cada 30 segundos del lado de dolarapi.

Como se usa en el flujo

1. User pide depositar \$10 USD
2. Backend lee rate blue.venta del cache (si no hay, fetchea y cachea 1h)
3. Calcula: $\text{amount_ars} = 10 * \text{rate_venta} = 10 * 1200 = \12.000 ARS
4. Devuelve al frontend: `{ amount_usd: 10, amount_ars: 12000, rate: 1200, source: "dolarapi.com - 17/06 18:00" }`
5. Frontend muestra: "Vas a pagar \$12.000 ARS (cotizacion blue)"
6. User confirma y paga via MP
7. MP notifica via webhook que pago \$12.000 ARS
8. Backend valida el monto (puede haber fluctuado 1%, tolerancia) y acredita 10 USD

Cache de rates

Tabla `rates_cache` con (currency, type, buy, sell, source, fetched_at). Refresh cada 1h. Si dolarapi cae, usamos el ultimo valor conocido y loggeamos warning. Para el user final, la UI siempre muestra el rate actual con timestamp.

10. Fase 2 - MercadoPago Argentina

Cuando implementar

Cuando vos ya no quieras acreditar a mano. Trigger: si te llega mas de 5 depositos por día, o si los 12 amigos se quejan de la fricción. Tiempo estimado: 2-3 días.

Setup de MP

1. Crear cuenta MercadoPago (ya la tenes)
2. Ir a <https://www.mercadopago.com.ar/developers/panel/credentials>
3. Crear aplicación "TNSVT Wallet"
4. Copiar credenciales:
 - APP_USR-xxxx (Access Token producción)
 - APP_USR-yyyy (Public Key)
5. Configurar webhook URL en el panel: <https://tnsvt.com/api/wallet/deposit/webhook/mp>
6. Agregar a .env:

```
MP_ACCESS_TOKEN=APP_USR-xxxx
MP_PUBLIC_KEY=APP_USR-yyyy
MP_WEBHOOK_URL=https://tnsvt.com/api/wallet/deposit/webhook/mp
```

SDK PHP oficial

```
Composer: composer require mercadopago/sdk:^2.5
Uso básico (Checkout Pro):
use MercadoPago\SDK\MercadoPago;
use MercadoPago\SDK\Preference;
use MercadoPago\SDK\Item;

MercadoPago::setAccessToken(env('MP_ACCESS_TOKEN'));
$preference = new Preference();
$item = new Item();
$item->title = 'T.N.S.V.T Wallet Deposit $10 USD';
$item->quantity = 1;
$item->unit_price = 12000; // ARS
$preference->items = [$item];
$preference->back_urls = [
    'success' => 'https://tnsvt.com/wallet?status=success',
    'failure' => 'https://tnsvt.com/wallet?status=failure',
    'pending' => 'https://tnsvt.com/wallet?status=pending',
];
$preference->notification_url = env('MP_WEBHOOK_URL');
$preference->external_reference = 'deposit_' . $depositId;
$preference->save();
return new JsonResponse(["checkout_url" => $preference->sandbox_init_point]);
```

Webhook handler

MP envía un POST al webhook cuando el pago se acredita. Estructura del payload:

```
{
  "type": "payment",
  "data": {
    "id": 123456789
  }
}
```

El handler hace GET a /v1/payments/{id} con el access token para obtener detalles, valida que status=approved, valida el monto, y acredita el wallet. Tambien valida la firma HMAC del header x-signature para evitar webhooks spoofeados.

Metodos de pago que acepta MP Argentina

Metodo	Tipo	Comision MP	Tiempo de acreditacion
Tarjeta de credito	Online	~3-6%	Inmediato
Tarjeta de debito	Online	~2-3%	Inmediato
Mercado Pago (saldo)	Online	0%	Inmediato
Rapipago / Pago Facil	Efectivo	~3-4%	24-48h (cuando paga)
Transferencia bancaria	Bancaria	0%	Inmediato
Cripto (via MP)	N/A	No soporta directo	-

Idempotencia del webhook

MP puede enviar el mismo webhook multiples veces. Por eso:

1. Antes de acreditar, hacemos SELECT por ref_payment_id en wallet_transactions
2. Si ya existe con status=confirmed, retornamos 200 sin hacer nada (idempotente)
3. Si no existe, creamos con status=confirmed y acreditamos el wallet
4. Si falla el credito, dejamos status=pending y reintentamos

11. Fase 3 - Binance Pay

Por que Binance Pay

Binance Pay es un sistema de pagos entre usuarios de Binance, sin fees, con crypto (USDT, USDC, BTC, ETH, BNB). Esta disponible en 100+ paises incluyendo Argentina, Mexico, Brasil, Colombia, Chile, Peru, etc. Para vos significa: tus amigos que ya estan en crypto pueden depositar sin friccion. Y vos recibis USDT en tu cuenta Binance, que podes cambiar a ARS via P2P o transferir a un exchange local.

Setup de Binance Pay

1. Crear cuenta Binance Business: <https://merchant.binance.com/>
2. Verificar identidad (KYC business - LDN/email/etc)
3. Activar Binance Pay en el panel
4. Crear API key con permisos de Pay
5. Copiar credenciales:
 - API Key
 - Secret Key
 - Merchant ID
6. Configurar webhook: <https://tnsvt.com/api/wallet/deposit/webhook/binance>
7. Agregar a .env:

```
BINANCE_PAY_API_KEY=xxx
BINANCE_PAY_SECRET=xxx
BINANCE_PAY_MERCHANT_ID=xxx
BINANCE_PAY_WEBHOOK=https://tnsvt.com/api/wallet/deposit/webhook/binance
```

SDK PHP

```
Composer: composer require binance/binance-connector-php:^1.0
Uso basico (crear orden de pago):
$api = new \Binance\Spot(['apiKey' => env('BINANCE_PAY_API_KEY'), 'apiSecret' =>
env('BINANCE_PAY_SECRET')]);
$response = $api->payCreateOrder([
    'merchantId' => env('BINANCE_PAY_MERCHANT_ID'),
    'merchantTradeNo' => 'deposit_' . $depositId,
    'totalFee' => '10.00',
    'currency' => 'USDT',
    'productName' => 'T.N.S.V.T Wallet Deposit',
    'productType' => '01',
    'transType' => 'APP',
    'returnUrl' => 'https://tnsvt.com/wallet?status=success',
    'cancelUrl' => 'https://tnsvt.com/wallet?status=cancel',
]);
return new JsonResponse(["checkout_url" => $response['data']['checkoutUrl'], "qrcode" =>
$response['data']['qrCodeLink']]);
```

Flujo del user

1. User clickea "Depositar con Binance Pay" en TNSVT
2. Backend crea la orden y devuelve QR + URL
3. User escanea QR con Binance app o abre URL
4. User confirma el pago en Binance (10 USDT)
5. Binance notifica via webhook
Payload: { merchantTradeNo, status: 'PAID', totalFee, currency }
6. Backend valida, acredita 10 USD al wallet (1:1 con USDT)

7. User ve el balance actualizado en su app

Ventajas vs MercadoPago

Caracteristica	MercadoPago AR	Binance Pay
Fee	2-6% segun metodo	0%
Moneda	ARS solamente	USDT, USDC, BTC, ETH, BNB
Países	Argentina (10 mas en LATAM)	100+ países
Tiempo de acredit.	Inmediato / 24-48h	Inmediato
KYC user	Si (CBU/cuenta)	No (ya lo tiene en Binance)
Para vos	Recibis ARS en tu MP	Recibis USDT en tu Binance
Cambio a ARS	Ya esta en ARS	P2P o exchange local

12. Fase 4 - Otros paises

Mapa de soluciones por pais

Pais	Recomendado Fase 2-3	Alternativa	Notas
Argentina	MercadoPago	Binance Pay / USDT	MP es lo mas comodo para ARS
Mexico	MercadoPago Mexico / Binance Pay	Conekta (OXXO, SPEI)	OXXO efectivo en 18k tiendas
Brasil	MercadoPago Brasil / Binance Pay	PIX directo	PIX es instantaneo 24/7
Colombia	MercadoPago Colombia / Binance Pay	Nequi / Daviplata	PSE para bancos
Chile	MercadoPago Chile / Binance Pay	Webpay / Khipu	Webpay es el mas usado
Peru	MercadoPago Peru / Binance Pay	Yape / Plin	Yape es de BCP
Uruguay	MercadoPago Uruguay / Binance Pay	PREX / MiDinero	PSE no aplica
USA	Stripe / Coinbase Commerce	PayPal	ACH + cards
Canada	Stripe / Coinbase Commerce	Interac	Interac e-Transfer
UK	Stripe / Wise	PayPal	FCA regulated
Europa SEPA	Wise / Stripe	SEPA Direct	1-2% fee
Asia / Africa	Crypto (USDT)	Binance Pay	Sin sistema bancario formal
Cualquier lado	USDT/USDC	-	Funciona siempre, sin KYC

Solucion universal: crypto directo (self-custody)

Para maxima cobertura sin depender de PSPs, una wallet self-custody multi-chain:

1. Backend genera una direccion unica por user, por chain
2. User envia USDT/USDC desde cualquier wallet (Binance, MetaMask, Phantom, Trust, etc)
3. Backend detecta el deposito via:
 - **Alchemy webhooks** (Ethereum, Polygon) - \$0-50/mes segun volumen
 - **BSCscan / TronGrid API** (BSC, Tron) - gratis con API key
 - **Block native** - alternativa paga
 - **Polling propio** cada 1 min via Alchemy/nownodes - \$0-50/mes
4. Cuando se detecta TX con confirmaciones ≥ 3 , se acredita el wallet del user
5. User ve el balance actualizado en su app

Pro: 100% cobertura global, sin KYC, 0% fee de PSP, solo gas de blockchain (<\$1)

Con: UX mas compleja (user tiene que entender crypto), riesgos de clave privada, requiere node RPCs

Chains recomendadas y sus fees

Chain	Token	Gas tipico	Tiempo confirmacion	Mejor para
Tron (TRC20)	USDT	\$0.50	60 segundos	Usuarios retail, bajo costo

Chain	Token	Gas tipico	Tiempo confirmacion	Mejor para
BSC (BEP20)	USDT/USDC	\$0.30	15 segundos	Velocidad + bajo costo
Polygon	USDC	\$0.005	5 segundos	Micropagos
Ethereum (ERC20)	USDT/USDC	\$3-10	5-15 min	Liquidez maxima
Solana	USDC	\$0.001	5 segundos	Lo mas rapido y barato

Cuando implementar cada uno

Fase 4A (mes 2): USDT-TRC20 + USDT-BEP20. Cubre 90% de usuarios crypto. Implementacion: 3-5 dias.

Fase 4B (mes 3): Polygon (USDC) + Ethereum (USDT). Para usuarios DeFi. Implementacion: +2 dias.

Fase 5 (cuando tengas 50+ users): Stripe (cards internacionales). 1 dia.

Fase 6 (cuando quieras expansion EU/UK): Wise (bancos). 1 dia.

Mi recomendacion: arranca con Fase 1 manual esta semana, Fase 2 MP la semana que viene, Fase 3 Binance Pay en 2 semanas, y Fase 4 crypto cuando ya tengas flujo constante de depositos (te dara tiempo para pensar bien la seguridad de la wallet self-custody).

13. Compliance & legal en Argentina

Disclaimer: No soy abogado. Esto es orientativo. Para temas legales serios, consulta con un profesional matriculado en Argentina.

Estado actual del proyecto

TNSVT es un juego didactico de trading con un Portfolio virtual. NO hay dinero real en el sistema. Los XP/levels/leaderboard son virtuales. Esto lo posiciona como:

- **Videojuego** (no juego de azar)
- **Software educativo** (no plataforma de inversion)
- **Servicio gratuito** (no opera dinero de terceros)

Regulado por: **Ley 25.675** (industria del software), no por Loterias ni BCRA.

Que cambia al agregar torneos con dinero real

Si los 12 amigos se ponen de acuerdo para jugar con plata, estas en un area gris:

- **Apuestas privadas** entre amigos: generalmente OK, no regulado
- **Pool de premios**: la plata se reparte segun habilidad, no suerte pura
- **Habilidad vs suerte**: como es trading (conocimiento), es mas skill que azar

La clave legal: NO sos una plataforma publica de juegos de azar. Sos un software de practica que ademas tiene un feature opcional de competencia entre usuarios.

Recomendaciones legales operativas

Tema	Recomendacion	Costo
Tipo societario	Monotributo (hasta \$7M anual)	~\$30k ARS/mes
Facturacion	Factura C a cada user por entry fee + payout	Tiempo admin
IIBB (ingresos brutos)	Segun jurisdiccion (CABA ~3%)	Autocalculado
Ganancias	Si superas monotributo, pasar a Resp Inscripto	Contador
PSPs (MP, Binance)	Ya estan registrados en BCRA como PSPs	\$0
UIF (Unidad Info Financiera)	Reportes solo si hay sospecha de lavado	\$0
KYC user	No obligatorio en Fase 1 (12 amigos)	\$0
KYC user	Obligatorio si escalas a publico (>100 users)	Onboarding
Terminos y condiciones	Disclaimers: simulacion, no inversion real, sin garantia	Abogado ~\$50k ARS
Proteccion de datos	Ley 25.326 - no compartir datos sin consent	\$0

Disclaimers obligatorios en la UI

En TODA pantalla relacionada a torneos/wallet:

- **Simulacion con dinero virtual.** Los precios son reales pero el dinero es virtual.
- **No es consejo financiero.** Competicion didactica entre traders.
- **Sin garantia de resultados.** El mejor trader puede perder.
- **Para mayores de 18.** Verificar KYC si se escala.

■ **Politica de reembolso:** el admin puede cancelar un torneo y devolver entry fees.

Cuando profesionalizar

Trigger: mas de \$500 USD/mes en entry fees o mas de 30 users.

Ahi si:

1. Habla con un contador (no abogado) para temas impositivos
2. Decide si queres SA, SAS, o monotributo
3. Implementa KYC basico (DNI + selfie) para users nuevos
4. Genera facturas automaticas con AFIP (via Facturapi o similar)
5. Reportes mensuales de UIF si aplica

Costo estimado: \$100-200k ARS/mes de contador + sistemas. Vale la pena si los ingresos lo justifican.

14. Plan por etapas (4-6 stages)

Las 10-12 horas de trabajo se dividen en 6 stages verificables. Cada stage termina con un test funcional antes de seguir. El user (vos) verifica y aprueba antes del siguiente.

Stage 1 - Backend foundation (~2.5h)

Entrega: DB migrada con 4 tablas nuevas + 4 entities + 1 controller simple

Detalle:

- Migration: add wallet_balance to users, create wallet_transactions, tournaments, tournament_entries
- Entities: User (+walletBalance), WalletTransaction, Tournament, TournamentEntry
- Repository para cada entity
- DolarController con GET /api/wallet/rates (dolarapi.com integration)
- Test curl: GET /api/wallet/rates devuelve blue, oficial, mep

Verificacion: php bin/console doctrine:migrations:migrate + curl rates endpoint

Stage 2 - Wallet system (~2.5h)

Entrega: 4 endpoints de wallet + 1 admin credit endpoint funcionales

Detalle:

- WalletController: balance, transactions, withdraw request
- AdminController: POST /api/admin/wallet/credit con X-Admin-Password
- AdminController: lista pending withdrawals, approve/reject
- Validaciones: wallet_insufficient, user_not_found, etc
- Test curl completo: acreditar -> ver balance -> ver transactions

Verificacion: acreditar \$10 a user de prueba, ver balance sube, ver tx en historial

Stage 3 - Torneos CRUD + cron (~3h)

Entrega: 6 endpoints de torneos + comando cron auto-close

Detalle:

- TournamentController: list, get, join, leaderboard, my (5 endpoints publicos/user)
- AdminController: create, close, cancel (3 endpoints admin)
- Logica de join: descuenta entry fee, captura starting_equity, crea tx wallet
- Logica de leaderboard: calcula pnl_pct en vivo desde PORT data
- Logica de close: ordena por pnl_pct, distribuye prize pool segun distribution
- Comando tournaments:process (corre via cron cada 1 min)
- Test: crear torneo -> 3 users join -> manipular PnL -> close -> ver winners

Verificacion: end-to-end test con 3 users de prueba, ver payouts correctos

Stage 4 - TNSVT main frontend (~2h)

Entrega: Sidebar con 2 tabs nuevos + UI completa de wallet y torneos + admin panel

Detalle:

- Sidebar: agregar botones "■ Mi Wallet" y "■ Torneos"
- Tab Wallet: balance, ARS equivalente, transacciones, boton depositar
- Tab Torneos: sub-tabs Activos/Mis/Historial, grid de cards, modal detalle

- Tab Admin: sub-tabs Wallet + Torneos con forms de acreditar y crear
- Llamadas API via api.js existente (window.API)
- Toasts de feedback (success/error)

Verificacion: visual en navegador, crear torneo desde admin, ver en user

Stage 5 - Game frontend (~1.5h)

Entrega: s-torneos refactorizado con data real del backend

Detalle:

- tournamentsInit() al abrir s-torneos: GET /api/tournaments/active
- tournamentJoin(id): POST con X-Game-Code, actualiza balance local
- tournamentLeaderboard(id): GET cada 30s mientras esta abierto
- UI: cards de torneos, modal de detalle, leaderboard inline
- Edge case: si no configuro TNSVT_SYNC, mensaje de ayuda
- Sincronizar via npx cap sync android

Verificacion: entrar al Game, ver torneos reales, joinear uno, ver rank en vivo

Stage 6 - Polish + deploy (~1h)

Entrega: APK reconstruido, doc actualizada, commit + push, landing page actualizada

Detalle:

- build_game_apk.bat -> APK nuevo en Downloads
- Actualizar landing page /download/tnsvt-market con info de torneos
- Actualizar boton sidebar TNSVT: link a "Torneos"
- git add -A + commit + push con mensaje descriptivo
- Commit history: feat(wallet), feat(torneos), chore(cron), chore(deploy)
- Resumen final para user con todos los endpoints + como probar

Verificacion: APK instalado, todos los flujos probados end-to-end

15. Costos, ROI y proyeccion

Costos del proyecto

Item	Costo	Notas
Desarrollo (vos haces)	\$0 (tiempo)	10-12 hs, ya cubierto
Hosting TNSVT (actual)	\$0	Esta en tu PC con Tailscale
Hosting produccion (Hostinger VPS)	\$9.99/mes	Recomendado en Etapa 2 de hosting
Dominio .com	~\$10/ano	Para que los 12 amigos entren por URL bonita
MP comision	2-6%	Solo cuando se usa MP
Binance Pay comision	0%	Gratis siempre
Contador (cuando factures)	~\$100-200k ARS/mes	Solo si >\$500 USD/mes en fees
Abogado (TyC + compliance)	~\$50k ARS one-time	Solo si escalas a publico

ROI con 12 amigos

Escenario conservador (1 torneo/semana, \$5 entry, 8 participants):

- 8 users x \$5 = \$40 USD prize pool por torneo
- 4 torneos/mes = \$160 USD prize pool/mes
- Tu margen: si cobras un 10% de rake = \$16 USD/mes

Escenario medio (2 torneos/semana, \$10 entry, 12 participants):

- 12 users x \$10 = \$120 USD prize pool por torneo
- 8 torneos/mes = \$960 USD prize pool/mes
- Tu margen: 10% rake = \$96 USD/mes

Escenario alto (1 torneo diario, \$15 entry, 12 participants):

- 12 users x \$15 = \$180 USD prize pool por torneo
- 30 torneos/mes = \$5.400 USD prize pool/mes
- Tu margen: 10% rake = \$540 USD/mes

Nota: el rake es opcional. Si no cobras rake, todo va al prize pool, y el valor para los users es mayor (motiva a participar). Podes cobrar en otros features (subscription \$9/mes).

Proyeccion de crecimiento

Mes	Users activos	Torneos/mes	Volumen USD	Notas
Mes 1	12	4	\$160	Lanzamiento MVP manual con amigos
Mes 2	25	8	\$640	MP Argentina integrado, llega +invitados
Mes 3	50	15	\$2.250	Binance Pay agregado, llegan crypto users
Mes 6	150	40	\$12.000	Crypto directo, scaling + influencer share

Mes	Users activos	Torneos/mes	Volumen USD	Notas
Mes 12	500+	120+	\$60.000+	Multi-pais, suscripciones \$9 + torneos

Conclusion

Con 12 amigos ya tenes un MVP validable. La implementacion es de 10-12 horas divididas en 6 stages verificables. Arrancamos con Fase 1 (manual) esta semana, automatizamos con MP y Binance en las siguientes 2 semanas, y dejamos crypto + Stripe + Wise para cuando el volumen lo justifique. El codigo de Fase 1 se mantiene - solo cambia el "como se acredita" de manual a automatico. Legalmente, en uso privado entre amigos no hay mayores restricciones, pero hay que sumar disclaimers y, cuando factures mas de \$500 USD/mes, hablar con un contador.

Proximo paso: arranco con Stage 1 (Backend foundation). Decime dale y voy.

Fin del documento.
Cualquier duda, preguntar antes de empezar.
Generado: 17/06/2026 16:24:47